Solución ejercicio acercamiento al rol Ingeniería de Datos

Pipeline de ingesta por micro-batches en AWS

https://github.com/ingenieroedgargalvis/retopragma

1. Resumen de la solución

Construí un pipeline que ingiere archivos CSV por micro-batches y mantiene estadísticas en ejecución (conteo, promedio, mínimo y máximo del campo price). El pipeline procesa un archivo a la vez y lee cada archivo fila a fila, de modo que la huella de memoria es constante y la solución escala a volúmenes grandes sin cambios.

Las estadísticas se mantienen con acumuladores incrementales (suma, conteo, mínimo, máximo); el promedio se deriva de suma/conteo. En ningún momento se recalcula con AVG() sobre toda la tabla, que es la restricción central del reto.

2. Interpretación del reto

El dataset de prueba es pequeño a propósito: representa un escenario de Big Data donde la información llega en fracciones y la capacidad de cómputo es limitada. Por eso la prueba no evalúa volumen que realice, sino el diseño: procesar con memoria acotada y mantener métricas sin recorrer de nuevo los datos ya cargados.

- Se cargan los 2012-1.csv … 2012-5.csv, excluyendo validation.csv.
- Nunca hay más de un archivo en memoria, y dentro de cada archivo se lee fila a fila.
- Las estadísticas se actualizan de forma incremental tras cada fila/archivo.
- validation.csv se procesa al final por el mismo pipeline, para comparar el antes y el después.

3. Arquitectura en AWS

La solución la desplegue en AWS con Terraform (infraestructura como código). El flujo es: los CSV se cargan en S3; una función Lambda los lee desde S3, calcula las estadísticas incrementales y escribe en una base de datos RDS PostgreSQL. La Lambda obtiene la credencial de la base desde Secrets Manager.

La base de datos es privada (sin acceso público). El acceso del operador se hace por un túnel SSH a través de una instancia EC2 que actúa como bastion, usando PgAdmin4. La Lambda vive dentro de la VPC y alcanza Secrets Manager y S3 mediante VPC endpoints, sin salir a internet.

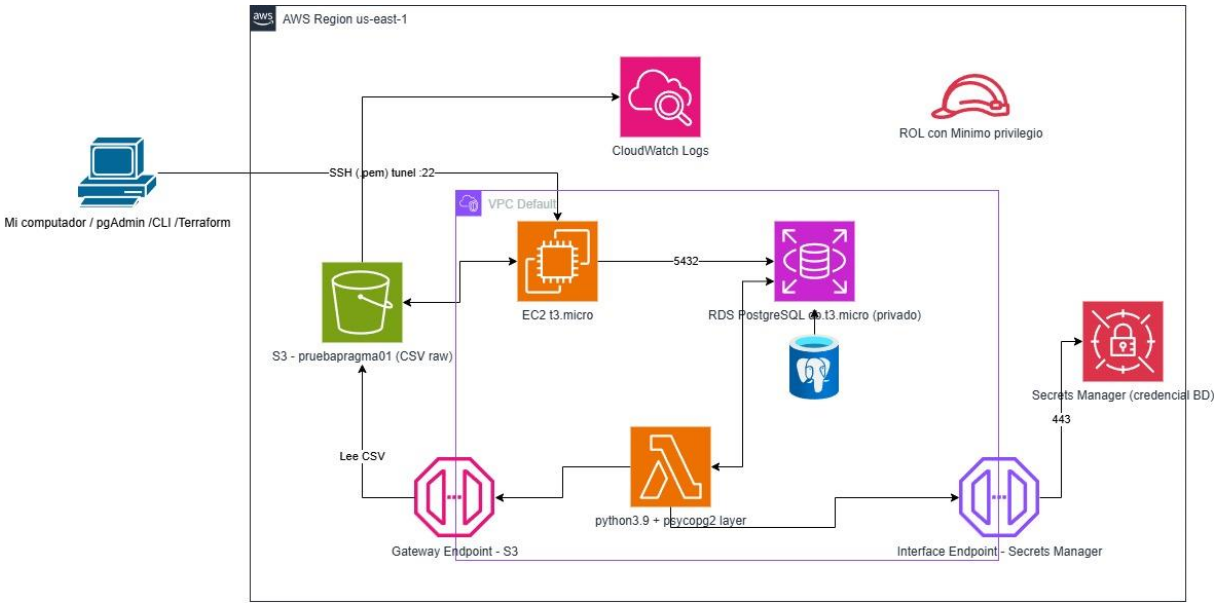


Figura 1. Arquitectura del pipeline en AWS.

Componentes

- S3 (pruebapragma01): zona raw de los CSV. Cifrado, versionado y sin acceso público.
- Lambda (python3.9): ejecuta el pipeline; incluye una layer propia con psycopg2.
- RDS PostgreSQL (db.t3.micro): base de datos destino, privada, Single-AZ.
- EC2 (t3.micro): bastion para el túnel SSH de PgAdmin4.
- Secrets Manager: almacena la credencial de la base de datos.
- VPC Endpoints: Interface para Secrets Manager y Gateway para S3.
- IAM: rol de la Lambda con privilegios mínimos (logs, lectura de S3, VPC y el secreto).

Despliegue con Terraform

El apply crea toda la infraestructura y expone los datos de conexión como outputs:

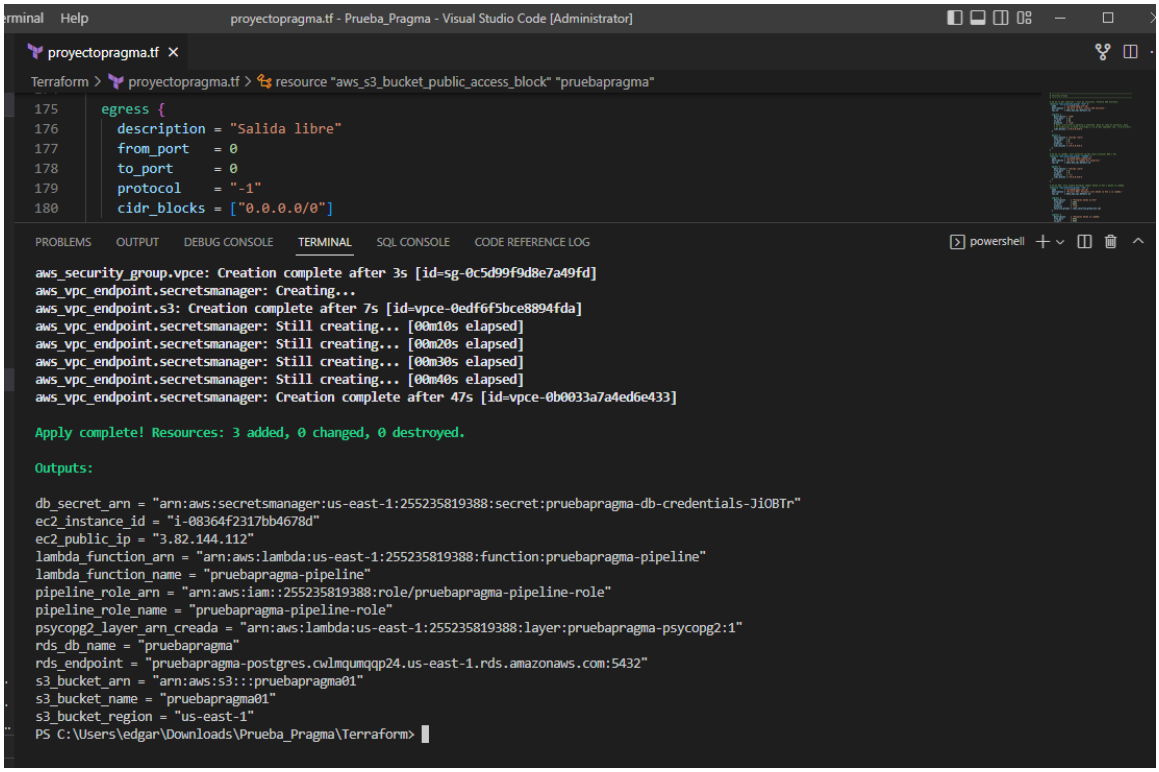


Figura 2. terraform apply completado con los outputs de los recursos.

4. Modelo de datos

El esquema vive en archivos SQL y se aplica aparte del pipeline. Dos tablas:

- transactions: los datos crudos (event_date, price, user_id) más auditoría de carga.
- running_stats: una sola fila con el estado incremental (row_count, sum_price, min_price, max_price).

El pipeline actualiza running_stats por delta tras cada micro-batch y puede reanudar su estado leyendo esa fila, de modo que las estadísticas en memoria y las de la base de datos siempre coinciden.

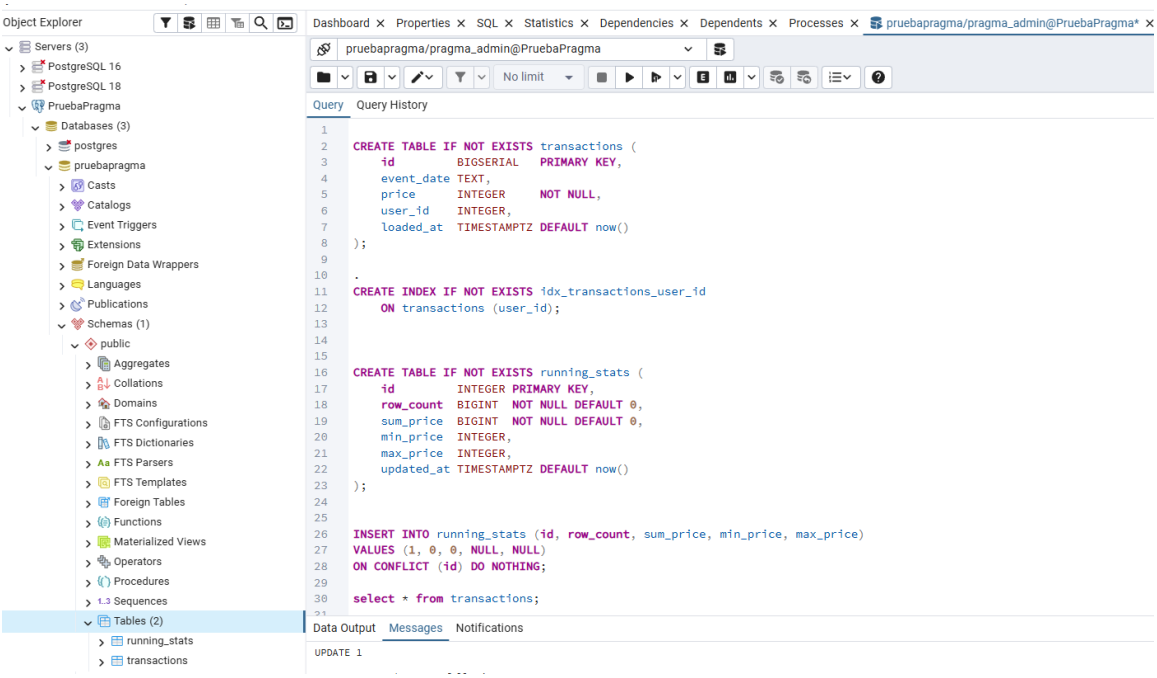


Figura 3. Creación de las tablas en PgAdmin4 (conexión vía túnel SSH).

5. Ejecución del pipeline

La Lambda se invoca con payload {} para procesar los archivos 2012-*.csv y con {"key": "validation.csv"} para la fase de validación.

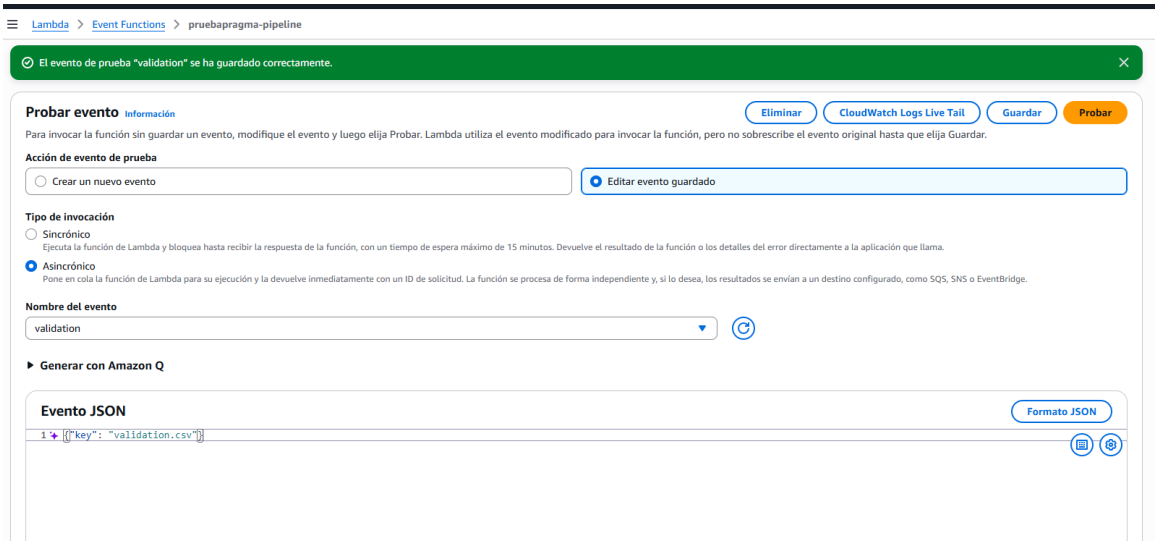


Figura 4. Configuración del evento de prueba para validation.csv.

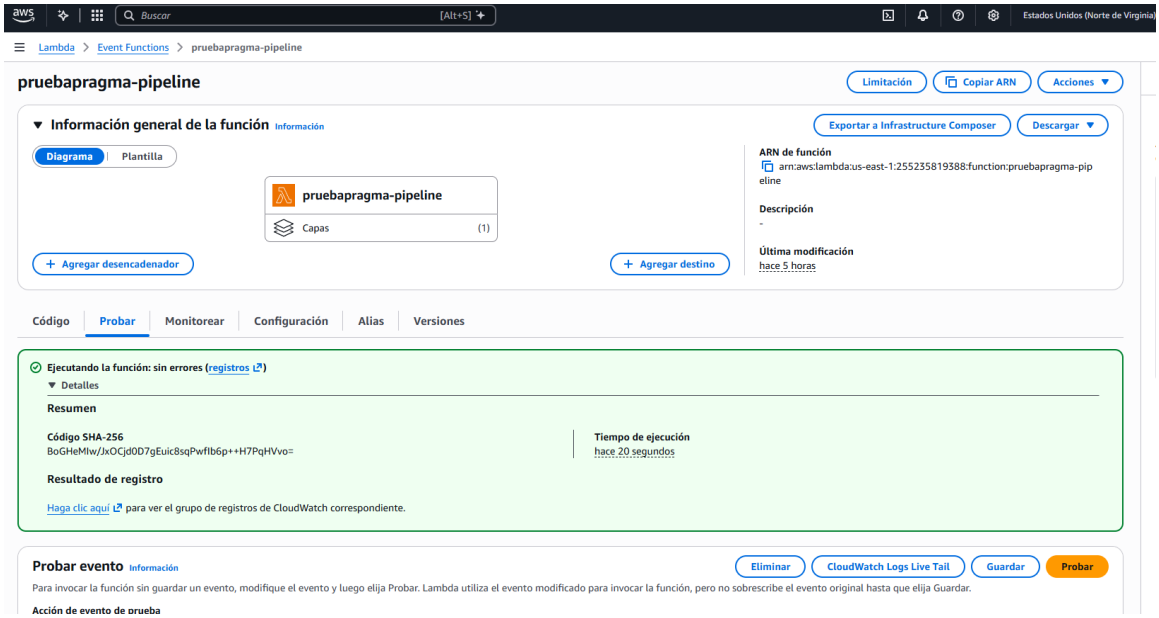


Figura 5. Ejecución de la Lambda finalizada sin errores.

6. Comprobación de resultados

La verificación compara el estado incremental (running_stats) contra un recálculo con funciones de agregación sobre la tabla transactions. Ambos deben coincidir en cada momento.

Resultados

Momento	Conteo	Promedio	Mínimo	Máximo
Tras los 2012-*.csv	139	57.8849	10	100
Tras validation.csv	147	57.0068	10	100

Tras procesar los archivos 2012-*.csv

```
34
35
36  -- Estado incremental guardado por el pipeline
37     SELECT row_count,
38            CASE WHEN row_count > 0 THEN sum_price::numeric / row_count END AS avg_price,
39            min_price,
40            max_price
41     FROM running_stats
42     WHERE id = 1;
43
44  -- Verificacion recalculando desde la tabla (
45     SELECT COUNT(*)   AS total_filas,
46            AVG(price) AS precio_promedio,
47            MIN(price) AS precio_minimo,
48            MAX(price) AS precio_maximo
49     FROM transactions;
50
51  -- Ejemplo de normalizacion de fecha a tipo DATE (capa de transformacion):
```

Data Output | Messages | Notifications

Showing rows: 1

	row_count bigint	avg_price numeric	min_price integer	max_price integer
1	139	57.8848920863309353	10	100

Figura 6. running_stats (estado incremental): 139 filas, promedio 57.8849.

```
34
35
36  -- Estado incremental guardado por el pipeline
37     SELECT row_count,
38            CASE WHEN row_count > 0 THEN sum_price::numeric / row_count END AS avg_price,
39            min_price,
40            max_price
41     FROM running_stats
42     WHERE id = 1;
43
44  -- Verificacion recalculando desde la tabla (
45     SELECT COUNT(*)   AS total_filas,
46            AVG(price) AS precio_promedio,
47            MIN(price) AS precio_minimo,
48            MAX(price) AS precio_maximo
49     FROM transactions;
50
```

Data Output | Messages | Notifications

	total_filas bigint	precio_promedio numeric	precio_minimo integer	precio_maximo integer
1	139	57.8848920863309353	10	100

Figura 7. Recálculo sobre transactions: coincide (139, 57.8849, 10, 100).

Tras procesar validation.csv

```
32  select * from running_stats;
33
34
35
36  -- Estado incremental guardado por el pipeline
37     SELECT row_count,
38            CASE WHEN row_count > 0 THEN sum_price::numeric / row_count END AS avg_price,
39            min_price,
40            max_price
41     FROM running_stats
42     WHERE id = 1;
43
44  -- Verificacion recalculando desde la tabla (
45     SELECT COUNT(*)   AS total_filas,
46            AVG(price) AS precio_promedio,
47            MIN(price) AS precio_minimo,
48            MAX(price) AS precio_maximo
49     FROM transactions;
50
```

Data Output | Messages | Notifications

Showing rows: 1 to

	row_count bigint	avg_price numeric	min_price integer	max_price integer
1	147	57.0068027210884354	10	100

Figura 8. running_stats tras validation: 147 filas, promedio 57.0068.

```
34
35
36  -- Estado incremental guardado por el pipeline
37     SELECT row_count,
38            CASE WHEN row_count > 0 THEN sum_price::numeric / row_count END AS avg_price,
39            min_price,
40            max_price
41     FROM running_stats
42     WHERE id = 1;
43
44  -- Verificacion recalculando desde la tabla (
45     SELECT COUNT(*)   AS total_filas,
46            AVG(price) AS precio_promedio,
47            MIN(price) AS precio_minimo,
48            MAX(price) AS precio_maximo
49     FROM transactions;
50
```

Data Output | Messages | Notifications

Showing rows: 1 to 1

	total_filas bigint	precio_promedio numeric	precio_minimo integer	precio_maximo integer
1	147	57.0068027210884354	10	100

Figura 9. Recálculo sobre transactions: coincide (147, 57.0068, 10, 100).

En ambos momentos el estado incremental coincide con el recálculo desde la base de datos, confirmando que las estadísticas se mantuvieron correctamente sin recalcular sobre todo el conjunto de datos.

7. Decisiones de diseño y evolución

Se optó deliberadamente por una solución simple y acorde al volumen: Python en streaming más PostgreSQL, en lugar de un motor distribuido. Para este régimen de datos, herramientas como Spark o Glue serían sobredimensionadas y además chocarían con la restricción de cómputo acotado que plantea el enunciado.

Ante un escenario near real-time a gran escala, la misma lógica de acumuladores incrementales evolucionaría hacia Kinesis para la ingesta de eventos, S3 en formato Parquet como data lake (vía Firehose) y Glue para el procesamiento histórico por lotes.